Course Code: ITA1439 **Course Name: Ethical Hacking**

Slot: A

CO2 – ASSIGNMENT_2

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: BP Venkata Uday Kumar **Reg. No: 192411203**

CO2: Execute footprinting and social-engineering techniques such as DNS analysis, competitive intelligence gathering and attack simulations to collect information for identifying potential security risks.

Assignment Title: Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Authorization, Scope and Lab Environment

The assignment follows the chain Understand → Predict → Design → Collect → Analyse → Evaluate → Defend → Validate. Each question below is written in that order. **All collection was performed only against lab-owned targets that I built myself.** No live system, no real organisation and no real person was queried or attacked, and no social-engineering attack was performed.

Lab component	What it is	Address / location
Authoritative DNS server	BIND 9.18 serving 13 lab zones under the reserved .example top-level domain (RFC 2606), which can never exist on the real Internet	127.0.0.1:5300
Recursive caching resolver	Second BIND instance that forwards to the lab server; used to study caching and “passive-style” DNS lookups	127.0.0.1:5301
WHOIS service	Small local server that returns lab-written WHOIS records; the real WHOIS system is never contacted	127.0.0.1:4343
Search engine and document corpus	Local search program that supports the operators site:, filetype:, intitle:, intext:, quoted phrases and exclusion, over 23 lab-written documents (job advertisements, annual reports, press releases, profiles) and an archive of old page snapshots. All people are synthetic	labsearch.py
Scale zone	bigcorp.example: 125 units × 6 hosts (750 records) used for Q6 and Q7	zone file
Tools	dig and nslookup (BIND 9.18 utilities), whois client, Python 3 with dnspython	local

Authorization statement. I am the owner of the lab and I authorized every query in this report. Zone transfers were attempted only on lab zones. The permitted scope list is the set of lab domains named in each question; anything else is treated as out of scope (Q1.2).

Prediction discipline. For Q1, Q2, Q3, Q4, Q5 and Q6 the predictions were written to a file before the collection script was run (predictions_q1_q2.json, predictions_q3.json, predictions_q4.json, predictions_q5.json, predictions_q6.json). The file modification time is kept as evidence. The predictions come from the design intent of each target, so a mismatch shows either a wrong assumption of mine or a defect in a script. Every mismatch is reported and explained instead of being hidden.

Q1 – Evaluate and Engineer a Footprinting Strategy (8 Marks)

1.1 Understand the problem

A single source gives an incomplete or misleading picture: a clean WHOIS record can hide DNS leakage, and a quiet DNS footprint can still be exposed in public documents. The strategy must therefore combine several sources, label each technique as passive or active, judge how reliable each source is, and keep out-of-scope, personal and unverified data apart from authorized, verified, inscope findings.

1.2 Footprinting strategy design

Classification rule. A technique is **passive** if the target's own servers cannot observe it (the traffic goes to a third party, a registry, a search index or a resolver). It is **active** if it sends traffic that the target's own infrastructure receives and can log. The strategy runs all passive techniques first and uses active ones only to verify or fill gaps, because active traffic is visible and costs effort.

ID	Technique	Tool / operator	Class	Why it is used
T1	Registration lookup	whois	Passive	Registrant, name servers, creation and update dates
T2	DNS records through a recursive resolver	dig @resolver NS, MX, TXT, A	Passive	The target's server sees only the resolver (and a cached answer may be old)
T3	Search-engine operators	site:, filetype:pdf, intitle:	Passive	Finds host names and contacts in public documents
T4	Archived pages	archive snapshots	Passive	Reveals old host names; likely outdated
T5	Authoritative host guessing	dig @authoritative <word>.<domain>	Active	Verifies names directly; target logs every query
T6	Zone-transfer attempt	dig <domain> AXFR (authorized only)	Active	Tests a serious misconfiguration
T7	Cross-source comparison	script	—	Detects contradictions and stale data

Rules that keep data apart. Every finding gets three tags before it is used:

- **Scope tag:** in-scope (name is inside the authorized domain), or out-of-scope (a CNAME to another organisation, a third-party service, or a name outside the domain). Out-of-scope names are recorded as boundary stubs and are never queried further.

- **Verification tag:** verified (answered by the authoritative server, or confirmed by a second independent source) or unverified (named by one document or archive only, or absent from DNS).
- **Sensitivity tag:** personal data (registrant names, personal e-mails, phone numbers, street address) is flagged, kept apart, and not used further.

Applied to the four targets, the recorded findings were classified as: 19 verified in-scope hosts, 4 unverified items (named in a document or archive but absent from DNS), 3 out-of-scope third-party CNAME targets and 5 personal-data fields (all in the leakyco.example WHOIS record). These groups are reported separately, never mixed.

1.3 Authorized test targets (four contrasting lab domains)

Target	Design intent	How it is built
shieldco.example	Well protected	Few records, transfer refused, strict SPF and DMARC (p=reject), privacy-redacted WHOIS, one press release only
leakyco.example	Leaks information	14 host names including vpn, jira, jenkins, git, staging; five TXT records revealing vendors and a helpdesk extension; WHOIS with registrant name, email, phone and street; documents naming hosts; third-party CNAMEs
misconfig.example	Misconfigured DNS	Zone transfer open to anyone; private 10.x addresses in public DNS; dangling CNAME to a removed service; SPF "+all"; MX to a host with no address; WHOIS still lists old name servers (last updated 2019)
minimal.example	Minimal public footprint	Apex record, one name server, one TXT record, no MX, no documents

1.4 Collection: commands and collected-source records

For each target the collector (q1_collect.py) ran this sequence, saving every raw output in the evidence folder:

```
whois -h 127.0.0.1 -p 4343 <domain>                    # T1
dig @127.0.0.1 -p 5301 <domain> NS | MX | TXT | A       # T2 via resolver
search: site:<domain>    and    site:<domain> filetype:pdf   # T3 archive
snapshot lookup                                        # T4
dig @127.0.0.1 -p 5300 <word>.<domain> A +short          # T5, 40 words dig
@127.0.0.1 -p 5300 <domain> AXFR                        # T6, lab zones only
```

Excerpt of raw output: TXT records of leakyco.example (T2)

```
$ dig @127.0.0.1 -p 5301 leakyco.example TXT +noall +answer leakyco.example.
293 IN TXT "MS=ms18273645"
leakyco.example. 293 IN TXT "support: IT helpdesk ext 4411, helpdesk@leakyco.example"
leakyco.example. 293 IN TXT "google-site-verification=LAB-xj29s8dk3" leakyco.example.
293 IN TXT "atlassian-domain-verification=LAB-77aa21" leakyco.example. 293 IN TXT
"v=spf1 mx include:_spf.mailprovider-saas.example include:crm-saas.example include:hr-
```

Excerpt of raw output: zone transfer of misconfig.example (T6)

```
$ dig @127.0.0.1 -p 5300 misconfig.example AXFR
; (1 server found) misconfig.example. 300 IN SOA ns1.misconfig.example.
hostmaster.misconfig.example.
2026110301 3600 600 86400 300 misconfig.example. 300 IN
MX 10 mail.misconfig.example. misconfig.example. 300 IN
MX 20 oldmail.misconfig.example.
misconfig.example. 300 IN TXT "v=spf1 +all"
misconfig.example. 300 IN NS ns1.misconfig.example.
misconfig.example. 300 IN NS ns2.misconfig.example.
misconfig.example. 300 IN A 192.0.2.210
admin.misconfig.example. 300 IN A 192.0.2.180
backup.misconfig.example. 300 IN A 10.10.6.5
db01.misconfig.example. 300 IN A 10.10.5.20
db02.misconfig.example. 300 IN A 10.10.5.21
dev.misconfig.example. 300 IN A 192.0.2.215
```

Collected-source records, organized by source

Source / item	shieldco.example	leakyco.example	misconfig.example	minimal.example
WHOIS name servers	ns1.shieldco, ns2.shieldco	ns1.leakyco, ns2.leakyco	ns1.oldhost, ns2.oldhost	ns1.minimal
WHOIS last updated	2026-09-30	2026-06-01	2019-01-15	2024-03-03
WHOIS personal data	No	Yes	No	No
DNS (resolver): name servers	ns1.shieldco, ns2.shieldco	ns1.leakyco, ns2.leakyco	ns1.misconfig, ns2.misconfig	ns1.minimal
DNS (resolver): MX present, TXT at apex	Yes, 1	Yes, 5	Yes, 1	No, 1

Search: documents found / host names in them	1 / –	3 / git, intranet, jira, staging, vpn	2 / ftp, old-portal, portal2, webmail	0 / –
Archive: host names	www	ftp, oldcrm, www	old-portal, portal2, webmail, www	–
Active: hosts found (40 words)	2 (www, mail)	9 (www, mail, vpn, dev, test, staging, intranet, git, ftp)	6 (www, mail, vpn, dev, admin, backup)	0
Zone transfer	Refused	Refused	Allowed, 18 records	Refused

Queries sent for this stage: whois 4, dns_resolver 16, search 8, archive 4, dns_active 168 (total 200).

1.5 Prediction versus observation

Item (predicted / observed)	shieldco.example	leakyco.example	misconfig.example	minimal.example
Zone transfer allowed	No / No	No / No	Yes / Yes	No / No
Hosts found by guessing	2 / 2	10 / 9 ✗	6 / 6	0 / 0
TXT records at apex	1 / 1	5 / 5	1 / 1	1 / 1
WHOIS personal data	No / No	Yes / Yes	No / No	No / No
WHOIS NS = DNS NS	Yes / Yes	Yes / Yes	No / No	Yes / Yes
Documents found	1 / 1	3 / 3	2 / 2	0 / 0
Source contradictions	0 / 0	1 / 1	3 / 4 ✗	0 / 0
MX present	Yes / Yes	Yes / Yes	Yes / Yes	No / No

Result: 30 of 32 predictions matched. The two mismatches (marked ✗) are explained below.

- **leakyco.example, hosts found by guessing: predicted 10, observed 9.** I assumed jira and webmail would be in the 40-word list. Neither is (the list holds common words such as www,

mail, vpn, git). The host jira was found only through a public document, which is exactly why one source is not enough.

- **misconfig.example, contradictions: predicted 3, observed 4.** I counted three host names that documents mention but DNS does not have (portal2, ftp, webmail) and forgot the fourth contradiction: the WHOIS record lists ns1/ns2.oldhost.example while DNS says ns1/ns2.misconfig.example.

1.6 Contradictions between sources

Target	Contradiction found
leakyco.example	host 'oldcrm' named in a document/archive but absent from DNS
misconfig.example	WHOIS NS ns1.oldhost.example, ns2.oldhost.example != DNS NS ns1.misconfig.example, ns2.misconfig.example
misconfig.example	host 'ftp' named in a document/archive but absent from DNS
misconfig.example	host 'portal2' named in a document/archive but absent from DNS
misconfig.example	host 'webmail' named in a document/archive but absent from DNS

1.7 Source-reliability analysis

Each source was scored against the authoritative DNS data (precision = share of its claims that are true) and by the age of its evidence.

Source	Claims	Correct	Precision (%)	Median age of evidence (days)
WHOIS (registry fixture)	4	3	75	565
Search documents	9	6	66.7	225
Archived pages	8	5	62.5	2743
DNS via resolver	18	18	100	0
DNS brute force (authoritative)	17	17	100	0

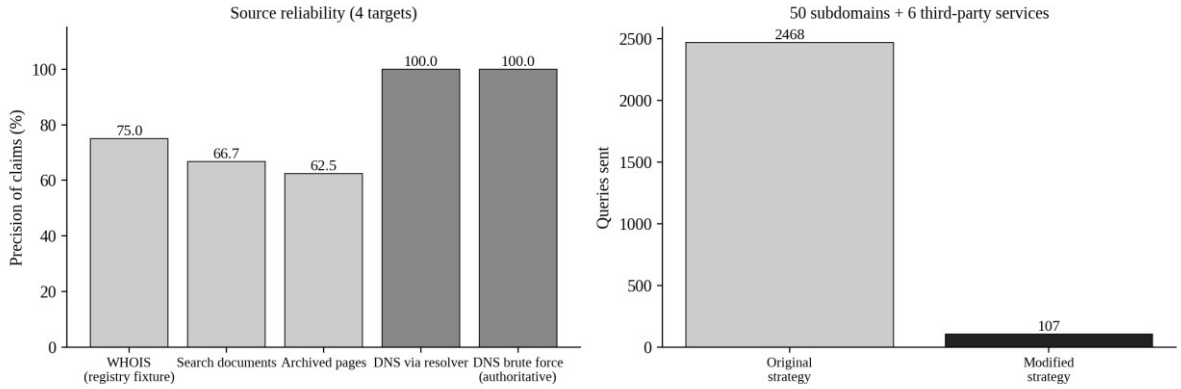


Figure 1: Source reliability (left) and query cost of the original and modified strategy at 50 subdomains (right)

- **Misleading or outdated sources.** Archived pages are the weakest (62.5 % precision, median age about 7.5 years) and WHOIS can be wrong: the misconfig.example record is from 2019 and names servers that DNS no longer uses. Search documents are 66.7 % correct because they name hosts that were retired or mistyped (portal2, ftp, webmail).
- **Reliable but not complete.** DNS answers from the authoritative server are 100 % correct but only for names the analyst thinks of. The resolver answer may also be a cached copy (shown in Q2, case C6).
- **Rule adopted.** A document or archive claim is only "verified" after a DNS lookup confirms it; WHOIS is used for ownership, never for current name servers.

1.8 Changed condition: scope grows to 50 subdomains and several third-party services

The lab zone example-corp.example holds 50 subdomains, six of which are CNAMEs to third-party services (status, support, shop, careers, events, blog). The original strategy (steps T1–T6 for every scope item) and a modified strategy were both run for real against the lab.

Measure	**Original strategy**	**Modified strategy**
WHOIS lookups	56	1
DNS queries via resolver	262	103
Guessing queries to authoritative server	2000	1
Zone-transfer attempts	50	0
Search queries	100	2
Total queries	**2468**	**107**
Queries sent to third-party names (out of scope)	18	0
In-scope items verified	44	44

Third-party services flagged as boundary	0 (6 wrongly treated as in-scope)	6
Local run time (supporting evidence only)	57.18 s	0.22 s

Evaluation. The original strategy is not suitable for 50 subdomains. It repeats the same WHOIS lookup 50 times for one registrable domain, repeats the NS/MX/TXT queries and a zone-transfer attempt for names that are not zones, guesses 40 words under every subdomain, and follows the six CNAMEs into other organisations (18 out-of-scope queries). It sent 2468 queries against 107 for the modified version, a reduction of 95.7 %, with the same verified in-scope result.

Modifications made: (1) normalise every scope item to its registrable domain and run WHOIS, NS, MX, TXT once; (2) query only A/CNAME per subdomain, in parallel (8 workers) with a rate limit; (3) treat a CNAME that leaves the authorized domain as a boundary stub and never query it; (4) one wildcard test name before any guessing; (5) attempt a zone transfer only if the authorization says so; (6) de-duplicate every item before it is recorded.

Is the strategy still suitable for large-scale authorized reconnaissance? Yes, after modification, with three limits. It verifies only names it is given or can find, so unlisted subdomains need guessing or certificate-transparency style sources; guessing cost grows with the wordlist and the number of domains (measured in Q6); and third-party services need their own authorization before any probing.

1.9 Final justified strategy

1. Confirm the scope list and mark every name as in-scope or boundary (third party).
2. Run passive sources T1–T4 once per registrable domain; record the evidence date.
3. Query the resolver for NS, MX, TXT once per domain and A/CNAME once per listed subdomain, in parallel and rate-limited.
4. Verify every document or archive claim with one DNS lookup; mark absent names as unverified or stale.
5. Run bounded active guessing only for the gap (Q7 shows how), and zone transfer only where authorized.
6. Tag every finding (scope, verification, sensitivity); report the three groups separately.

Evidence delivered for Q1: strategy design (1.2), authorized test targets (1.3), collected-source records (1.4), prediction-versus-observation (1.5), contradictions (1.6), source-reliability analysis (1.7), scope analysis (1.8) and the justified final strategy (1.9).

Q2 – Build a Reliable DNS Footprinting Workflow (7 Marks)

2.1 Understand the problem

DNS answers cannot be reported at face value: records may differ by viewpoint (split-horizon), be passed to another operator (delegation), be a cached copy, be stale, or be exposed by a zone transfer that should have been refused. The workflow therefore records not only the answer but also how it was obtained.

2.2 Workflow design (dig and nslookup)

```
1  dig  <domain> SOA +noall +answer        # serial, refresh, TTL: freshness clues
2  dig  <domain> NS  +noall +answer        # who is authoritative
3  dig  <domain> A | MX | TXT +noall +answer
4  nslookup -type=MX <domain> <server>     # cross-check with a second tool
5  dig  <name> A +norecurse                # referral? (AUTHORITY section, no "aa" flag)
6  dig  <domain> AXFR          # ONLY where the authorization names it; lab zones only
7  compare: resolver vs authoritative; external vs internal viewpoint; inventory vs DNS
8  interpret: tag each record (live, stale, delegated, cached, internal-address, dangling)
```

Step 6 is skipped for any domain not listed as “transfer permitted” in the authorization. In the lab it was attempted on every zone because I own them all.

2.3 Authorized test cases (seven contrasting lab cases)

Case	Zone	Configuration built	What it tests
C1	shieldco.example	Transfer refused; minimal records	Correctly secured server
C2	misconfig.example	Transfer allowed to anyone; internal addresses; dangling CNAME	Zone-transfer exposure
C3	stale.example	TTL 86400, SOA serial from 2021, two hosts decommissioned (lab inventory file)	Stale records
C4	split.example	BIND views: external viewers get 2 hosts, internal viewers (127.0.0.2) get 5	Split-horizon DNS
C5	deleg.example	sub and sub2 delegated to other name servers	Delegation (third-party boundary)
C6	cache.example	TTL 20 s; record changed on the server while a resolver holds the old answer	Caching
C7	tsig.example	Transfer allowed only with a TSIG key	Countermeasure

2.4 Prediction versus observation

Case	Quantity	Predicted	Observed	Match
C1 secure	axfr	REFUSED	REFUSED	Yes
C1 secure	std records exposed	5	5	Yes
Case	**Quantity**	**Predicted**	**Observed**	**Match**
C2 transfer	axfr	SUCCESS	SUCCESS	Yes
C2 transfer	distinct rrs in transfer	18	18	Yes
C3 stale	stale hosts	2	2	Yes
C3 stale	total hosts	3	3	Yes
C4 split-horizon	external hosts	2	2	Yes
C4 split-horizon	internal hosts	5	5	Yes
C5 delegation	www sub answer	REFERRAL	REFERRAL	Yes
C5 delegation	sub2 answer	REFERRAL	REFERRAL	Yes
C6 caching	resolver returns old after change	Yes	Yes	Yes
C6 caching	auth returns new immediately	Yes	Yes	Yes
C7 TSIG	axfr without key	REFUSED	REFUSED	Yes
C7 TSIG	axfr with key	SUCCESS	SUCCESS	Yes
C7 TSIG	distinct rrs in transfer	11	11	Yes
Countermeasure	axfr after	REFUSED	REFUSED	Yes
Countermeasure	residual hosts via bruteforce	6	6	Yes

Result: 17 of 17 comparisons matched in the final run. The first run matched only 13 of 17. The four disagreements were caused by my tooling and not by DNS: (i) rndc could not authenticate (the control channel used the wrong key), so the record change and the later transfer restriction never reached the server; (ii) dig prints the TSIG signature as a pseudo-record, which my parser counted as a zone record (12 instead of 11). After fixing the control key and excluding the TSIG pseudo-record from the count, the rerun matched all 17 and the raw outputs below are from that run.

2.5 What each query returns (trace)

Case	Query	Result observed

C1	dig AXFR shieldco.example	Transfer failed (refused). Standard queries expose 5 records (2 NS, 1 A, 1 MX, 1 TXT). nslookup MX agrees with dig.
C2	dig AXFR misconfig.example	Transfer succeeds: 18 distinct records. Standard queries expose only 6.
C3	dig AXFR stale.example + inventory	Hosts in DNS: www, crm, mx-old. Inventory says crm (decommissioned 2024-02) and mx-old (2023-11) are gone, so 2 of 3 are stale. SOA serial year 2021; TTL 86400 s.
C4	dig hr-db.split.example (external, then -b 127.0.0.2)	External viewer resolves www, mail (2 hosts). Internal viewer resolves www, mail, hr-db, fileserver, dc01 (5 hosts).
Case	**Query**	**Result observed**
C5	dig www.sub.deleg.example A +norecurse	NOERROR, ANSWER 0, AUTHORITY 1 (NS ns1.sub.deleg.example), no aa flag: a referral. sub2 behaves the same (REFERRAL, REFERRAL). A host in the parent zone gives a real ANSWER.
C6	resolver before / authoritative after change / resolver at once / resolver after 21 s	192.0.2.80 / 192.0.2.81 / 192.0.2.80 / 192.0.2.81
C7	dig AXFR tsig.example without key, then with key	Without key: REFUSED. With key: SUCCESS, 11 records.

Raw output excerpt: delegation referral (C5)

```
$ dig @127.0.0.1 -p 5300 www.sub.deleg.example A +norecurse
status: NOERROR   flags: qr   ANSWER: 0, AUTHORITY: 1
sub.deleg.example.   300  IN  NS  ns1.sub.deleg.example.
```

2.6 Effect of split-horizon DNS, delegation and caching on an external tester

- **Split-horizon:** an external tester sees 2 of the 5 hosts (40 %). The hidden hosts (hr-db, fileserver, dc01) are the most valuable ones, so an external result must never be reported as “the DNS footprint”. It describes only the external view. Internal data seen from outside (private 10.x addresses in C2) is a leak, not a feature.
- **Delegation:** the parent zone answers with a referral and the real records belong to another operator. The tester cannot see them without querying that operator, who may be a third party, so the delegation point is a scope boundary. Whether the delegated server answers (a “lame” delegation) cannot be tested without contacting it, so it is reported as “not tested – out of scope”.
- **Caching:** after the server changed the record, the resolver kept returning the old address for the full TTL and only returned the new one after the TTL expired (192.0.2.80 until 21 s later, then

192.0.2.81). A tester using a resolver therefore sees yesterday's DNS; the authoritative server must be asked for a current answer, and the TTL should be recorded with every record.

2.7 Does a misconfigured zone transfer meaningfully change the attacker's knowledge and risk?

Yes. Without the transfer, the attacker sees 6 apex records. With it, the attacker receives all 18 records, i.e. 12 named hosts, three times as many records, including everything that guessing cannot find.

Exposed by the transfer	Why it matters	Likelihood	Impact	Risk
db01, db02, backup with private addresses 10.10.x.x	Reveals internal network layout and database/backup hosts	High	High	High
admin host	Names the administration interface as a target	High	Medium	High
Exposed by the transfer	**Why it matters**	**Likelihood**	**Impact**	**Risk**
old-portal → removedapp.thirdparty-gone.example (dangling CNAME)	Opens a subdomain-takeover opportunity if the third-party name is registered by someone else	Medium	High	High
oldmail MX with no address; ns2 stale address	Mail routing and name-server weaknesses	Medium	Medium	Medium
Host names guessable anyway (www, mail, vpn, dev)	Little extra knowledge	High	Low	Low

2.8 Countermeasure and redesigned analysis

Two countermeasures were applied. (a) On misconfig.example the transfer was restricted to nobody (allow-transfer { none; }) and the server reloaded. (b) On tsig.example the transfer was restricted to holders of a TSIG key (HMAC-SHA256), which is the right design for legitimate secondary servers.

Measure	Before (transfer open)	After (transfer restricted)
Zone-transfer attempt	Success	Refused
Host names learned by an attacker	12	6 (by guessing 40 common words: www, mail, vpn, dev, admin, backup)
Private 10.x addresses exposed	3 (backup, db01, db02)	1 (backup)
TSIG zone: without key / with key	–	REFUSED / SUCCESS (11 records)

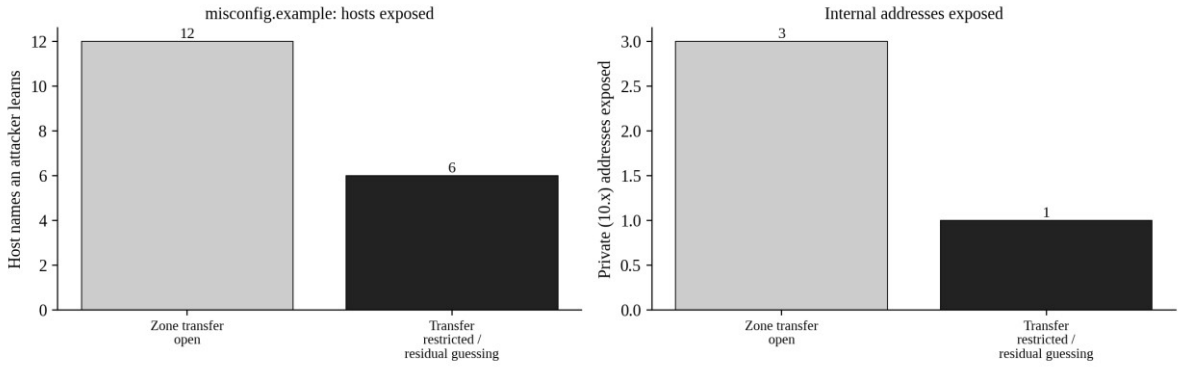


Figure 2: Exposure of misconfig.example before and after restricting zone transfers

Redesigned analysis. Restricting the transfer removed half of the host names (12 to 6) and two of three internal addresses (3 to 1), so exposure fell clearly but did not reach zero. The host "backup" (10.10.6.5) is still found because it is a common word. A complete countermeasure also needs: remove private addresses from public DNS (use split-horizon), avoid guessable names for sensitive hosts, delete the dangling CNAME, and use TSIG for authorized secondaries. TSIG only controls who may transfer; it does not hide records from the key holder.

Evidence delivered for Q2: DNS workflow (2.2), authorized test cases (2.3), record analysis and trace (2.5), prediction-versus-observation (2.4), effect of split-horizon, delegation and caching (2.6), zone-transfer risk evaluation (2.7) and countermeasure redesign (2.8).

Q3 – Create a Competitive-Intelligence Awareness Utility (7 Marks)

3.1 Understand the problem

Public business information (job advertisements, annual reports, press releases, public profiles) can help an attacker write believable phishing, vishing, pretexting and tailgating attempts. The utility collects that public information about an authorized organisation, maps each item to the technique it could enable and the warning sign a target should notice, and scores the human-risk so that awareness training and countermeasures can be designed. **No social-engineering attack is performed against anyone;** the three test organisations and all their staff are synthetic.

3.2 Workflow architecture (modular)

Module	Responsibility	How to extend
1 Source registry	A list of sources; each has search-operator templates and a document-type filter	Add one entry (no other change)
2 Search layer	Runs the operators (site:, filetype:, intitle:, quoted phrases) over the corpus and keeps provenance (URL, date, source)	None
3 Extractors	Pull items out of text: e-mail, e-mail format, phone, helpdesk extension, person and title, vendor/technology, schedule, physical-access details	Add a rule
4 Verifier	Gives each document a confidence value (stale, unverified, denied, look-alike domain, placeholder phone, contradicts known facts)	Add a rule
5 Technique mapper	Maps each item to phishing, vishing, pretexting or tailgating and to the indicator a target should notice	Edit one table
6 Risk scorer	Weight × confidence summed per source; Low < 6, Medium 6–14.99, High ≥ 15	Edit thresholds
7 Countermeasure layer	Reduces the weight of the item types a countermeasure acts on and recomputes the score	Add one entry

3.3 Documented steps and source operators

Source	Search operators (documented steps)	Provenance recorded for every item
Job advertisements	site:<domain> intitle:careers · site:<domain> intitle:jobs	URL, document date, source name, confidence
Annual reports	site:<domain> filetype:pdf	same

Press releases	site:<domain> intitle:press · site:<domain> intitle:home	same
Public profiles	site:profiles.example "<organisation name>"	same; profile must name the organisation
Source	**Search operators (documented steps)**	**Provenance recorded for every item**
Added later: conference talks	site:conf-talks.example intitle:talk	same
Added later: vendor case studies	site:vendor-cases.example intitle:case	same

Attribution rule: a document is attributed to an organisation only if it is on the organisation's own site or names the organisation. Items are de-duplicated inside a source.

3.4 Test organisations (rich, sparse, misleading)

Organisation	Design	Documents
Brightwave Logistics (brightwavelogistics.example)	Rich: tech stack, names and titles, vendors, e-mail format, phone and helpdesk extension, gates, shifts, travel and event dates	8 across the four sources
Tiny Foundry (tinyfoundry.example)	Sparse: one contact address and an empty jobs page	2
Orionworks (orionworks.example)	Misleading: a 2019 report naming a CEO who was replaced in 2024, a closed 2017 job ad, a denied partnership, a placeholder phone number, and a fake profile on a look-alike domain (orionw0rks.example) mentioning a gate that does not exist	5

3.5 Item-to-technique mapping

Item type	Technique enabled	Indicator a target should notice	Weight
E-mail address; e-mail address pattern	Phishing	Unexpected mail from a look-alike or internal-looking sender asking for action; mail to many staff using a guessed address pattern with urgency and credential links	2 / 3

Phone number; helpdesk extension	Vishing	A caller quotes a real name or number and asks for codes, passwords or payments; a “helpdesk” asks for an MFA code	3 / 4
Person with title; vendor or technology; schedule or event	Pretexting	A request that cites a real executive, vendor or project and bypasses the normal approval; a message timed to a real trip	2 / 3 / 2
Gate, badge, reception, shift pattern	Tailgating	An unknown person follows staff through a named gate or claims to be a contractor or visitor	4

Examples of collected items with provenance (Brightwave Logistics)

Source	Item	Technique	Where it came from
job ads	email: careers@brightwavelogistics.example	phishing	brightwavelogistics.example/careers/n
job ads	person: Anita Rao (Head of IT)	pretexting	brightwavelogistics.example/careers/n
annual reports	email: firstname.lastname@brightwavelogistics.example	phishing	brightwavelogistics.example/reports/ar 2025.pdf
annual reports	email_format: firstname.lastname@brightwavelogistics.example	phishing	brightwavelogistics.example/reports/ar 2025.pdf
press releases	email: dev.nair@brightwavelogistics.example	phishing	brightwavelogistics.example/news/nev
press releases	phone: +91.8000001002	vishing	brightwavelogistics.example/news/nev
profiles	email: anita.rao@brightwavelogistics.example	phishing	profiles.example/in/anita-rao
profiles	person: Anita Rao (Head of IT)	pretexting	profiles.example/in/anita-rao

3.6 Prediction versus observation

Organisation	Source	Predicted level	Observed level (score)	Match

brightwave-logistics	job ads	High	High (31)	Yes
brightwave-logistics	annual reports	High	High (43)	Yes
brightwave-logistics	press releases	Medium	High (17)	No
brightwave-logistics	profiles	High	High (21)	Yes
brightwave-logistics	most dangerous source	annual_reports	annual_reports	Yes
tinyfoundry	job ads	Low	Low (0)	Yes
tinyfoundry	annual reports	Low	Low (0)	Yes
tinyfoundry	press releases	Low	Low (2)	Yes
tinyfoundry	profiles	Low	Low (0)	Yes
tinyfoundry	most dangerous source	press_releases	press_releases	Yes
orionworks	job ads	Low	Low (2.4)	Yes
Organisation	**Source**	**Predicted level**	**Observed level (score)**	**Match**
orionworks	annual reports	Medium	Low (3.6)	No
orionworks	press releases	Medium	Low (0.6)	No
orionworks	profiles	Medium	Low (1.1)	No
orionworks	most dangerous source	annual_reports	annual_reports	Yes

Result: 11 of 15 predictions matched. The four mismatches are explained:

- **Brightwave press releases: predicted Medium, observed High (17.0).** The score is just above the High threshold of 15. I underestimated how much one release gives away (a named project lead, a partner, a press phone number and an event date).
- **Orionworks annual report, press releases and profiles: predicted Medium, observed Low.** I predicted from the amount of information, but the verifier discounts misleading data (stale, denied, look-alike domain, placeholder phone). Without the confidence step Orionworks would score 37.0 raw points; with it 7.7 (a 79.2 % reduction). The fake profile is still dangerous in practice, because an attacker does not need it to be true, so unverified items should be reported as “misleading exposure” and watched, not ignored.

3.7 Risk evaluation

Organisation	Items found	Raw score	Confidenceweighted score	Overall human-risk level
brightwave-logistics.example (rich)	40	112.0	112.0	High
tinyfoundry.example (sparse)	1	2.0	2.0	Low
orionworks.example (misleading)	14	37.0	7.7	Low

Does the information meaningfully increase social-engineering risk? For the rich organisation, yes. The 40 items map to 6 phishing, 4 vishing, 23 pretexting and 7 tailgating enablers. Combined, they allow a complete scenario: a caller names the Head of IT, mentions the ServiceNow helpdesk and the ERP project, uses extension 5500 and the Gate 3 contractor entrance, and then sends mail in the real address format. For the sparse organisation the risk is negligible (2 points), and for the misleading organisation it is small but not zero.

Most dangerous source: annual reports (score 43, 38.4 % of the total). They name the whole leadership, the vendors, the e-mail address format and the visitor and contractor entry rules in one document. Ranking: annual reports 43, job ads 31, profiles 21, press releases 17.

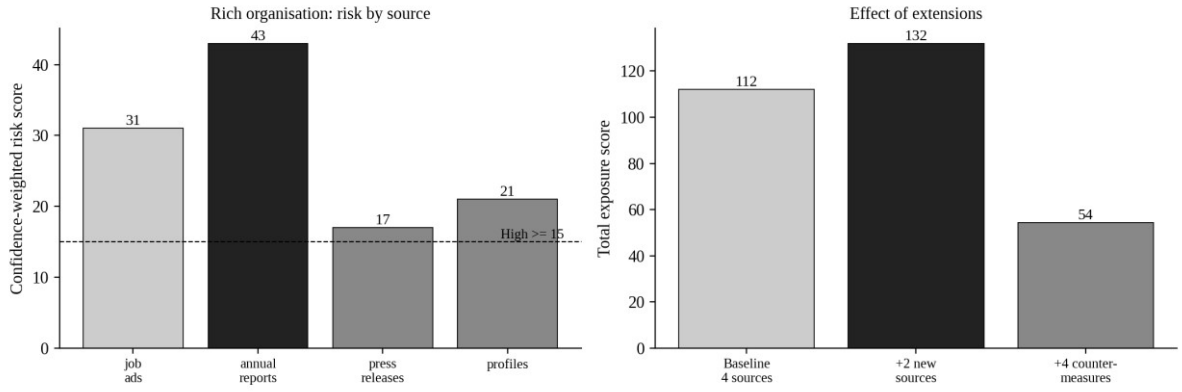


Figure 3: Risk by source (left) and the effect of two new sources and four countermeasures (right), rich organisation

3.8 Extension without restructuring: two new sources and four countermeasures

Two new sources (conference talks and vendor case studies) were added by two new lines in the source registry; the search layer, extractors, mapper and scorer were not touched. Conference talks alone score 9 (Medium); vendor case studies score 11 (Medium). The total exposure score of the rich organisation rose from 112.0 to 132.0 (+20.0). The vendor case study is the more serious addition because a vendor engineer's name, e-mail address and phone number are ideal material for pretexting. The tiny and misleading organisations did not change, which shows the attribution rule works.

Countermeasure (assumed effectiveness)	Acts on	Reduction in richorganisation score (points)
Call-back verification for helpdesk and finance requests (70 %)	phone, helpdesk_ext	9.1
External-sender banner and one-click phishing report button (50 %)	email, email_format	6.5
Badge check and visitor-escort rule at every gate (60 %)	physical	16.8
Publication review: no tech stack, extensions or shift details in public ads (60 %)	tech_vendor, schedule	25.2

Applied together, the four countermeasures reduce the rich organisation from 112.0 to 54.4 points (−51.4 %). **The effectiveness percentages are assumptions** that I chose for the model; they show relative value, not measured results. The ranking is robust: reducing what is published (CM4) and physical access controls (CM3) save the most points because they act on the heaviest item types.

3.9 Record of sources queried, items found and countermeasures recommended

Source	Queries per organisation	Items (rich / sparse / misleading)	Level (rich)
job ads	2	10 / 0 / 3	High
Source	**Queries per organisation**	**Items (rich / sparse / misleading)**	**Level (rich)**
annual reports	1	14 / 0 / 5	High
press releases	2	7 / 1 / 2	High
profiles	1	9 / 0 / 4	High
conference talks	1	3 / 0 / 0	Medium
vendor case studies	1	4 / 0 / 0	Medium

Recommended awareness countermeasure	Technique addressed	Target indicator taught
Review before publishing: remove technology names, extension numbers, shift and gate details from job adverts and reports	Pretexting, vishing, tailgating	Public text should not tell outsiders how to sound like an insider

Call-back rule: verify any request for passwords, codes or payments by calling a number from the directory	Vishing, pretexting	Caller quotes real names and is in a hurry
External-sender banner and one-click "report phishing" button	Phishing	Mail that looks internal but comes from outside
Badge check and escort rule at every gate; no "holding the door"	Tailgating	Unknown person following you or claiming to be a contractor
Vendor-contact policy: vendor staff are verified through the project manager before any request is honoured	Pretexting	Someone naming the right vendor and project but not on the approved list

Evidence delivered for Q3: workflow architecture (3.2), source operators (3.3), item-to-technique mapping (3.5), test organisations (3.4), prediction-versus-observation (3.6), risk evaluation (3.7), extension (3.8) and recommended countermeasures (3.9).

Q4 – Design a Reconnaissance Data-Validation Mechanism (6 Marks)

4.1 Understand the problem and design the mechanism

Every collected item (host, record type, value, source, date seen) must be validated before use. Four checks are applied, and each item ends in exactly one of four outcomes:

Check	Rule	Cost units	Failing outcome
Scope	After normalising (lower case, trailing dot removed, non-ASCII rejected) the host equals an authorized domain or ends with "." + that domain, matched on whole labels	1	REJECT (out of scope)
Duplication	Same (normalised host, type, value) already seen	1	DUP (dropped, not an error)
Freshness	Seen within the last 30 days	1	RECHECK (stale, recoverable)
Source reliability	Required fields present; source score ≥ 0.5 pass, 0.2–0.49 recheck, < 0.2 reject (dns_auth 1.0, whois 0.9, dns_resolver 0.9, search_doc 0.6, archive 0.4, paste_site 0.1)	3	RECHECK (needs confirmation) or REJECT (unverifiable)

Recoverable versus rejected. A **recoverable** item is real but uncertain (stale, incomplete, from a weak source) and is fixed by one re-query to confirm it. A **rejected** item can never be used: it is out of scope or comes from an unverifiable source, and no re-query may be sent for it. A re-query costs 10 units because it is a network action. The key safety requirement is that **no re-query is ever sent to an out-of-scope host**.

4.2 Adversarial test sequences

Sequence	Composition (items)	Purpose
S1 all valid	18 valid + 2 valid written as "WWW1.EXAMPLE-CORP.EXAMPLE." (upper case, trailing dot)	No false rejects
S2 out of scope	10 valid + 6 other-vendor hosts + 4 look-alikes (notexample-corp.example, …example.evil.example, examp1e, a Cyrillic letter)	Scope matching
S3 stale	12 fresh + 8 older than 30 days	Freshness and recovery
S4 duplicated	10 originals + 10 repeats (case, trailing dot, other source)	De-duplication
S5 mixed adversarial	47 items: valid, stale, weak and unverifiable sources, incomplete, out-of-scope that are also stale or weak, look-alikes, duplicates, duplicates of out-of-scope items	Order traps
S6 incomplete	10 valid + 6 with a missing value + 4 from an archive source	Recoverable items

4.3 Prediction versus observation

The prediction is the count of each correct outcome, derived from how each sequence was constructed. It was saved before the validator was run. The first version of the validator (v1) used the original order reliability → freshness → duplication → scope and a simple scope test (host ends with the domain name).

Sequence	Items	Predicted pass / recheck / reject / dup	v1 observed	v1.1 observed (scope fixed)	Out-ofscope accepted (v1 / v1.1)
S1 all valid	20	20 / 0 / 0 / 0	20 / 0 / 0 / 0	20 / 0 / 0 / 0	0 / 0
S2 out of scope	20	10 / 0 / 10 / 0	11 / 1 / 8 / 0	10 / 1 / 9 / 0	1 / 0
S3 stale	20	12 / 8 / 0 / 0	12 / 8 / 0 / 0	12 / 8 / 0 / 0	0 / 0
S4 duplicated	20	10 / 0 / 0 / 10	10 / 0 / 0 / 10	10 / 0 / 0 / 10	0 / 0
S5 mixed adversarial	47	10 / 12 / 20 / 5	10 / 21 / 9 / 7	10 / 21 / 9 / 7	0 / 0
S6 incomplete	20	10 / 10 / 0 / 0	10 / 10 / 0 / 0	10 / 10 / 0 / 0	0 / 0

- **Defect 1 (S2, v1): one look-alike host was accepted as in scope.** The simple test "host ends with the domain" accepts x.notexample-corp.example. Predicted 10 pass, observed 11. Fix (v1.1): match on whole labels (equal, or ends with "." + domain). This is a scope-breach defect, so it was fixed first.
- **Defect 2 (S2 and S5, v1.1): items that should be rejected as out of scope were sent for rechecking.** With the original order, an out-of-scope stale or weak item reaches the freshness or reliability check first and is labelled RECHECK; the scope check is never reached. S5: predicted 12 recheck / 20 reject, observed 21 / 9, and 9 re-queries would have been sent to out-of-scope hosts. S6, S3 and S4 matched exactly. This is an ordering problem, evaluated next.

4.4 Does the order of the checks matter? (all 24 orders)

All 24 orders of the four checks were run on the same 147 items (all six sequences, v1.1 scope test).

Order of checks	Accuracy (%)	Effort (units)	Checks run	Requeries	Re-queries to out-of-scope hosts
reliability → freshness → duplication → scope	91.8	1158	464	40	10
scope → duplication → freshness → reliability	100.0	943	461	30	0
duplication → scope → freshness → reliability	98.6	954	472	30	0
Order of checks	**Accuracy (%)**	**Effort (units)**	**Checks run**	**Requeries**	**Re-queries to out-of-scope hosts**
freshness → duplication → scope → reliability	94.6	1019	477	36	6
reliability → duplication → freshness → scope	91.8	1160	466	40	10

Findings. (1) The order changes accuracy: the original order labels 91.8 % of items correctly, the best order 100 %. (2) It changes effort mostly through re-queries, not through the number of checks (464 against 461): the original order spends 40 re-queries and 10 of them go to out-of-scope hosts. (3) Only 8 of the 24 orders send no re-query to an out-of-scope host, and every one of them puts the scope check first. (4) Placing duplication before scope changes the label of a repeated out-of-scope item from REJECT to DUP (accuracy 98.6 %) but is still safe.

4.5 Redesign of the validation order

Redesigned order: scope → duplication → freshness → reliability. Reasoning: scope is the cheapest check and the only safety-critical one, so it runs first and nothing outside the scope is ever examined, re-queried or accepted. Duplication is also cheap and removes repeats before the more

expensive checks. The 3-unit reliability check runs last, on the fewest items. Result on the combined data: accuracy 100 %, effort 943 against 1158 units (−18.6 %), re-queries 30 against 40, out-of-scope re-queries 0, out-of-scope items accepted 0.

Trade-off. When a sequence has no out-of-scope items or duplicates, the earlier checks do not save anything. In S6 the redesign costs 220 units against 190 for the original, because the scope and duplication checks are run on items that all pass. The cost is small and constant (2 units per item), and it is the price of the guarantee.

Sequence	Original accuracy (%)	Original effort	Original outof-scope requeries	Redesigned accuracy (%)	Redesigned effort
S1 all valid	100.0	120	0	100.0	120
S2 out of scope	95.0	128	1	100.0	70
S3 stale	100.0	184	0	100.0	176
S4 duplicated	100.0	110	0	100.0	80
S5 mixed adversarial	76.6	426	9	100.0	277
S6 incomplete	100.0	190	0	100.0	220

4.6 Scalability inquiry: one target to fifty targets

Expected behaviour, reasoned from the design before the run (this one was not saved to a file): the outcome counts and the effort grow in direct proportion to the number of targets (each target brings the same 47-item adversarial sequence), the original order keeps sending out-of-scope re-queries at the same rate, and the redesign keeps them at zero.

Targets	Items	Original: effort	Original: out-ofscope requeries	Original: accuracy (%)	Redesign: effort	Redesign: out-ofscope requeries	Redesign: pass / recheck / reject / dup
1	47	434	10	74.5	277	0	10 / 12 / 20 / 5
5	235	2170	50	74.5	1385	0	50 / 60 / 100 / 25
10	470	4332	99	74.7	2770	0	100 / 120 / 200 / 50
25	1175	10826	247	74.7	6925	0	250 / 300 / 500 / 125
50	2350	21700	500	74.5	13850	0	500 / 600 / 1000 / 250

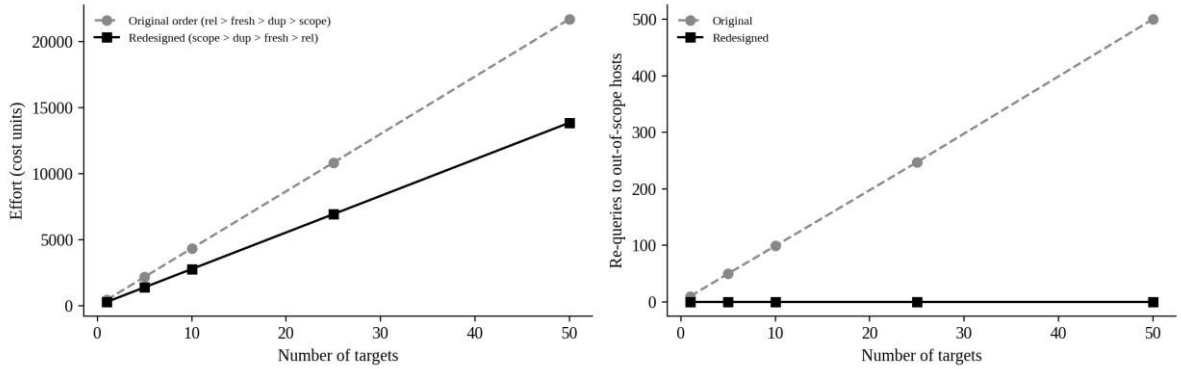


Figure 4: Validation effort (left) and out-of-scope re-queries (right) from 1 to 50 targets

Analysis. The prediction held. At 50 targets the redesign yields exactly 50 times the single-target outcome (500 / 600 / 1000 / 250). Effort is linear for both orders, but the redesign costs 5.9 units per item against 9.2, and the original order sends 500 out-of-scope re-queries at 50 targets (about ten per target), while the redesign sends none. Accuracy of the original order stays near 74.5 % at every scale, so scale does not repair an ordering defect.

Evidence delivered for Q4: validation design (4.1), adversarial inputs (4.2), pass/fail/re-check analysis (4.3), ordering evaluation (4.4), justified redesign (4.5) and scalability analysis (4.6).

Q5 – Evaluate Footprinting of a Hierarchical Organization (7 Marks)

5.1 Hierarchical representation

The public footprint is a tree. Each node has: a name, a kind (domain, subdomain, department, service, third_party, delegated_partner), an in_scope flag, the number of public information items found there, and a list of children. The root is the parent domain; depth is the number of steps from the root.

Test structure "irregular" (the most complex of the three) irr.example (root, 2 items)

```
├─ depta  (department)
│    ├─ a1, a2, a3  (each has 2 service leaves)          → 10 nodes
├─ deptb  (department)
│    └─ b1 → b2 → b3 [delegated partner: OUT OF SCOPE] → b4 (below the boundary)
├─ deptc  (department, EMPTY: no data, no children)
├─ mail   (leaf)
└─ saas   [third-party service: OUT OF SCOPE] → s1..s6 → 3 more nodes below (9 nodes)
```

Structure	Description	Total nodes
shallow	Root with 6 children (5 in-scope hosts and 1 third-party status page), depth 1	7

deep	Chain of 5 in-scope nodes (depth 0–4); the next node (e) is a third-party boundary with 3 nodes below it	9
irregular	Branches of different size and depth: a 10-node department, a chain cut by a delegated partner, an empty department, a leaf, and a 10-node third-party subtree	28

5.2 Systematic traversal with an explicit boundary condition

```
visit_ok(node, depth):  return node.in_scope AND depth <= MAXD        # BOUNDARY CONDITION
queue = [(root, 0)]
while queue:
   node, depth = pop_front(queue)              # level by level (breadth-first)
if not visit_ok(node, depth):
      record node.name as a BOUNDARY STUB; continue     # never probe, never expand
   record node in level[depth]; add node.items to the information count
for child in node.children: push_back(child, depth + 1)
terminal nodes (no children) stop naturally; an empty node is recorded, not treated as an error
```

Why breadth-first: it records each level completely before the next (the assignment asks for levelby-level recording), and under a limited budget it leaves the deepest, least certain levels for last. The boundary condition is checked before a node is visited, so a third-party or delegated node is named but never queried and its children are never added to the queue.

5.3 Prediction versus observation

Structure	Quantity	Predicted	Observed	Match
shallow	visited	6	6	Yes
shallow	boundary stubs	1	1	Yes
shallow	unreached	0	0	Yes
shallow	info items	7	7	Yes
shallow	levels	[1, 5]	[1, 5]	Yes
deep	visited	5	5	Yes
deep	boundary stubs	1	1	Yes
deep	unreached	3	3	Yes
deep	info items	5	5	Yes
deep	levels	[1, 1, 1, 1, 1]	[1, 1, 1, 1, 1]	Yes

irregular	visited	16	16	Yes
irregular	boundary stubs	2	2	Yes
irregular	unreached	11	10	No
irregular	info items	16	16	Yes
irregular	levels	[1, 4, 4, 7]	[1, 4, 4, 7]	Yes
irregular	empty branch handled	true	true	Yes

Result: 15 of 16 predictions matched. The one mismatch is unreached nodes in the irregular structure: predicted 11, observed 10. The error was mine: I counted the third-party node “saas” itself as unreached, but it is a boundary stub and was already counted as one. The third-party subtree below it has 9 nodes, plus b4 below the delegated partner makes 10. The traversal was right and the hand count was wrong.

Levels visited in the irregular footprint: depth 0: 1, depth 1: 4, depth 2: 4, depth 3: 7. Boundary stubs recorded: saas.irr.example, b3.irr.example.

5.4 Effect of depth and branching on effort and completeness

For a complete tree with branching factor b and depth d, the number of nodes is (b^(d+1) − 1)/(b − 1), so the effort grows geometrically with depth and with branching. With a budget of 100 nodevisits, a breadth-first traversal completes whole levels only:

Branching b	**Depth d**	**Nodes**	**Share of nodes in fully completed levels (%)**
2	2	7	100
2	4	31	100
2	6	127	49.6
Branching b	**Depth d**	**Nodes**	**Share of nodes in fully completed levels (%)**
3	2	13	100
3	4	121	33.1
3	6	1093	3.7
4	2	21	100
4	4	341	24.9
4	6	5461	1.6

6	2	43	100
6	4	1555	2.8
6	6	55987	0.1

Reading the table. Depth hurts more than branching at small sizes, but both matter: a tree with b = 3 and d = 4 (121 nodes) is only 33 % covered by whole levels within the budget, and b = 6, d = 6 (55,987 nodes) is practically out of reach. In irregular footprints a single wide branch consumes most of the effort, so the boundary condition matters more than the traversal order: cutting off the 10-node third-party subtree saved 10 of 28 visits (36 %) and removed the out-of-scope risk.

5.5 Empty branch and node with no public data

The department “deptc” has no data and no children. The traversal visited it at depth 1, recorded it with 0 information items, did not expand it, and raised no error (empty nodes handled: 1). The boundary condition correctly distinguishes **empty** (in scope, nothing found: record and stop) from **out of scope** (not in scope: name only, never probe). An empty node is real information (“nothing public here”), so it is kept in the result.

5.6 Systematic traversal compared with an ad-hoc approach

The ad-hoc analyst was modelled as someone who follows any link, keeps no list of visited nodes (revisits with probability 0.3) and has no scope test. Each ad-hoc run was given the same number of node-visits as the systematic traversal and repeated 500 times. **This is a simple model of ad-hoc behaviour, so the numbers show the direction, not exact values.**

Structure	Systematic: coverage (%)	Ad-hoc: mean coverage (%)	Ad-hoc runs with full coverage (of 500)	Systematic: out-ofscope touches	Ad-hoc: mean outof-scope touches	Ad-hoc: mean repeated visits
shallow	100	77.9	47	0	0.79	0.6
deep	100	84.1	195	0	0	0.8
irregular	100	51.2	0	0	5.82	2.13

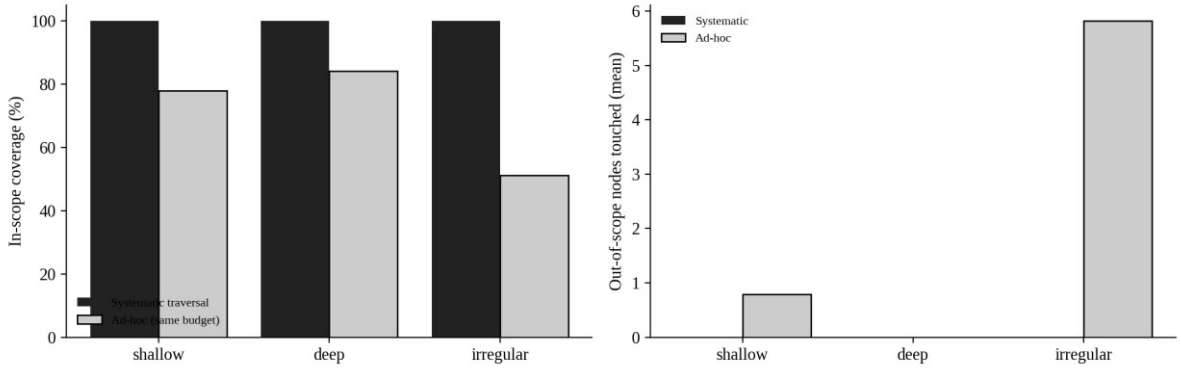


Figure 5: Systematic traversal compared with an ad-hoc approach under the same budget

Conceptual comparison. A systematic traversal gives complete coverage of the in-scope tree, zero out-of-scope contact, no repeated work and a record of every level. The ad-hoc approach is cheaper to start but its coverage is unpredictable (51 % on the irregular footprint, with no run reaching full coverage), it repeats work, and it touched about six out-of-scope nodes per run on the irregular footprint, which in a real assessment would be unauthorized queries. Its only advantage is flexibility, which the systematic design keeps through the depth limit and an optional priority order inside a level.

5.7 Verification and final design

- **Shallow, deep and irregular footprints are all covered**: every in-scope node was visited exactly once (6, 5 and 16 visits) and all expected information counts matched (7, 5, 16).
- **No out-of-scope node was probed**: the four boundary stubs (status in the shallow footprint, e in the deep one, saas and b3 in the irregular one) are named only; none of their descendants entered the queue (0 out-of-scope visits).
- **Final design:** breadth-first traversal, boundary test before the visit, empty and terminal nodes recorded without error, a depth limit, and a per-level log. The one change made after testing is to compute the "unreached" count from the traversal itself rather than by hand.

Evidence delivered for Q5: hierarchical representation (5.1), traversal design (5.2), test structures with predictions versus observations (5.1, 5.3), boundary analysis (5.5, 5.7), depth and branching evaluation (5.4), ad-hoc comparison (5.6) and the justified final design (5.7).

Q6 – Evaluate Footprinting Approaches Under Constraints (7 Marks) 6.1 Approach definitions

	Approach A: passive open-source collection	**Approach B: active DNS footprinting**
What it does	Reads public documents, archives and registration data about the target; asks a search index, not the target	Sends DNS queries directly to the target's authoritative server (guessing host names from a 150-word list, one NS query, one zone-transfer attempt)
What it produces	Candidate host names with a source and a date (not verified; may include stale and third-party names)	Verified host names that exist right now, with their addresses

What the target sees	Nothing	Every query is written to its DNS log
Typical weakness	Stale, repeated and out-of-scope items; misses unpublished hosts	Many wasted queries; misses hosts whose names are not in the word list; noisy

6.2 The two workflows

A for each unit: 1 search query (site:<unit> ...) # + 1 WHOIS lookup for the parent
collect candidate names from every document source (same name may repeat)
report candidates (unverified) ; record the source of each
B for each unit: for each of 150 words: dig @auth <word>.<unit>.<domain> A keep names that answer ; one NS query and one AXFR attempt for the parent

6.3 Test design (same authorized target for both approaches)

The target is the lab zone bigcorp.example. A scope of S units means the first S units of the zone. Each unit has 6 real hosts: **3 common names** (in the first 30 words of the list and also published in documents), **2 semi-guessable names** such as project names (published in documents but not in the list), and **1 obscure name** (in the last 10 words of the list and not published). The public documents also contain, for each unit, **1 stale host** that no longer exists in DNS and **1 third-party name** that is out of scope, and the same host is often repeated by two or three sources. Four scope sizes were tested (1, 5, 25 and 125 units); the practical limits are 6,000 queries and 10 minutes at a polite rate of 10 queries per second.

6.4 Quantitative results

Scope (units)	A: queries	A: coverage / precision (%)	A: lines in target log	B: queries	B: coverage / precision (%)	B: lines in target log	B: time measured	B: time at 10 q/s	B practical?
1	2	83.3 / 83.3	0	152	66.7 / 100.0	152	0.08 s	0.3 min	Yes
5	6	83.3 / 83.3	0	752	66.7 / 100.0	752	0.24 s	1.3 min	Yes
25	26	83.3 / 83.3	0	3752	66.7 / 100.0	3752	0.95 s	6.3 min	Yes
125	126	83.3 / 83.3	0	18752	66.7 / 100.0	18752	4.88 s	31.3 min	**No**

At 1,000 units, approach B would need 150,002 queries (about 4.2 hours at 10 queries per second), while approach A would need 1,001. This projection was calculated from the measured per-unit cost

and was not run. Time is shown only as supporting evidence; the main evidence is coverage, reliability and query counts.

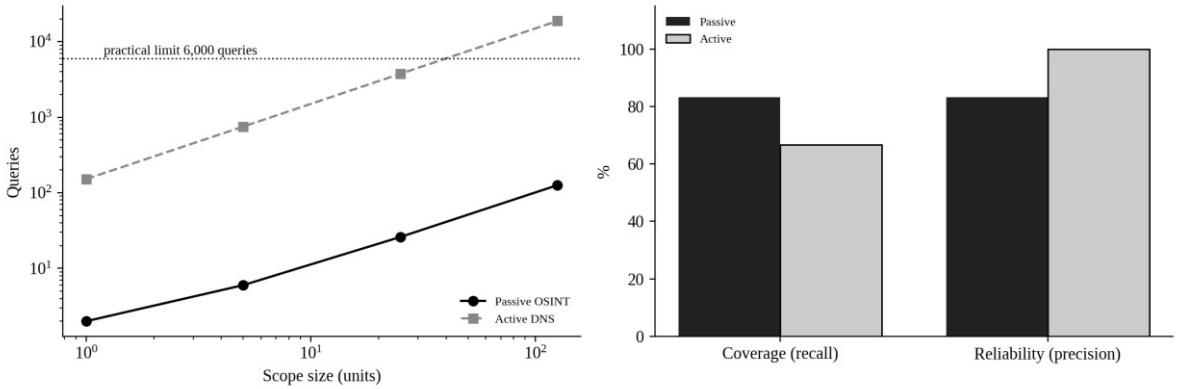


Figure 6: Queries against scope size (left; both axes logarithmic) and coverage and reliability at 25 units (right)

6.5 Prediction versus observation

Quantity	Predicted	Observed	Match
Passive: queries = S + 1	S + 1	2, 6, 26, 126	Yes
Passive: coverage (recall)	83.3 %	83.3 %	Yes
Passive: precision	83.3 %	83.3 %	Yes
Passive: lines in target log	0	0	Yes
Passive: practical at 125 units	Yes	Yes (126 queries)	Yes
Active: queries = 150 × S + 2	150 S + 2	152, 752, 3,752, 18,752	Yes
Quantity	**Predicted**	**Observed**	**Match**
Active: coverage (recall)	66.7 %	66.7 %	Yes
Active: precision	100 %	100 %	Yes
Active: lines in target log	= queries	= queries at every size	Yes
Active: practical at 125 units	No	No (18,752 > 6,000)	Yes
Scope size where an approach becomes impractical	125	125	Yes

Result: 11 of 11 predictions matched. This is expected because the lab was built from the same design, so the value of this table is that it confirms the counts are exact and that the target's own DNS log recorded every active query (e.g. 18,752 lines for 18,752 queries), proving that the active approach is fully visible.

6.6 Redundancy analysis: where effort is wasted

Approach	Redundant or repeated collection	Evidence (125 units)
B Active	The same 150-word list is run against every unit although 110 of the words (73 %) are filler that exist in no unit. Almost every query returns “no such name”.	18,250 of 18,752 queries (97.3 %) found nothing; 37.5 queries per host found
B Active	No memory of what is already known: names published in documents are guessed again; no link between units	375 of the 500 hosts found (the 3 common names of every unit) were already named in public documents
A Passive	The same host is reported by two or three sources (repeated collection); stale and thirdparty items are re-reported for every unit	1,629 records for 750 unique items: 754 repeats (46.3 %); 125 out-of-scope items reported; 125 stale items
Both	Overlap between approaches: 3 of the 6 hosts of every unit are found by both	375 hosts (3 per unit) are found by both approaches

6.7 Is the faster or the quieter approach always the better engineering choice?

No. The passive approach is both faster and quieter at every scope size, but its output is 16.7 % wrong (stale and out-of-scope items) and it misses hosts that were never published. The active approach is slower, loud and wasteful, but every item it reports is true. Which is better depends on what the assessment needs:

Condition	Better choice	Reason from the measurements
Stealth is required (the client wants to test detection, or must not alert operators)	Passive, then stop	Zero lines in the target log; accept 83 % precision and flag unverified items
Condition	**Better choice**	**Reason from the measurements**
Scope is very small (1–5 units) and the client has authorized active testing	Active (or passive + full guessing)	Only 152–752 queries; finds the obscure host that passive misses; 100 % coverage is possible (Q7)
Scope is large (25 units or more)	Passive first, then verify and guess in a bounded way	Active alone needs 3,752 to 18,752 queries and still misses the semiguessable hosts
The target has no public documents	Active	Passive finds nothing (Tiny Foundry in Q3); active still finds 66.7 %
Evidence quality matters (report will be used for decisions)	Active verification of every passive item	Only DNS answers are 100 % reliable (Q1)

6.8 Defended recommendation

- **Small-scope engagement (1–5 units):** run the passive step, verify every candidate with one query each, then run the full word list. For 5 units this costs 765 queries and gave 100 % coverage with 100 % precision (Q7). If stealth is required, stop after the passive step.
- **Large-scope engagement (25 units and more):** passive first, scope-filter and de-duplicate, verify the candidates, then guess only a short list of common words (top 30) while skipping names already verified. At 125 units this needs 4,125 queries (within the 6,000 limit) against 18,750, with higher coverage (83.3 % against 66.7 %).
- **Teaching use:** run both approaches on the 1-unit and 5-unit lab so that students can see the counts, the lines in the target log, the stale and out-of-scope items in the passive result and the hosts that each approach misses. The cost is only a few hundred queries, and the lesson is the comparison itself.
- **Repeated real assessments:** use the redesigned hybrid with a cache of verified names, deduplication, a scope filter and a rate limit. Pure active guessing would repeat the same 97 % wasted queries every time.

Evidence delivered for Q6: approach definitions (6.1), the two workflows (6.2), quantitative measures (6.4), redundancy analysis (6.6) and a defended recommendation (6.8).

Q7 – Integrated Review: Diagnose, Redesign and Defend (8 Marks)

Selected workflow: the active DNS footprinting workflow of Q6 (approach B). It shows a meaningful limitation (excessive queries and incomplete coverage) that can be measured exactly because the lab's DNS log counts every query. **A. Diagnose the Limitation (2 Marks)**

Evidence from Q6 (125 units)	Value
Queries sent	18,752
Queries that found nothing	18,250 (97.3 %)
Queries per host found	37.5
Time at a polite 10 queries per second	31.3 minutes
Practical limit (6,000 queries)	Exceeded 3.1 times
Coverage of real hosts	66.7 % (the 2 semi-guessable hosts of every unit are never found)
Lines written to the target's DNS log	18,752

Bottleneck: the guessing step. It is the only step whose cost grows with scope (150 queries per unit) and almost all of that cost buys nothing.

Possible cause	Evidence	Verdict

Redundant collection	97.3 % of queries return nothing; the same 150 names are tried against every unit; names already published in documents are guessed again	**Primary cause**
Source unreliability	Every reported host is confirmed by the authoritative server (100 % precision)	Not a cause
Poor ordering	There is only one step; the order of the steps does not change the count	Not a cause
Risk of going out of scope	All queries stay inside the authorized zone; but the workflow has no scope filter, so it would also query any third-party name it is given	Latent risk, fixed in the redesign

Underlying cause: redundant collection together with the lack of memory and of prior knowledge. The workflow does not use what the passive sources already say, so it spends its budget guessing.

B. Establish Evaluation Criteria (1 Mark)

Criterion	**How it is measured**	**Target for the redesign**
Coverage	% of the real hosts found (ground truth in the lab zone)	Not lower than the original (66.7 %)
Criterion	**How it is measured**	**Target for the redesign**
Number of queries	Queries sent to the authoritative server (counted, and confirmed from the server log)	70 % fewer than the original at 25 and 125 units
Source reliability	Precision (% of reported items that are real) and number of unverified items reported	100 % precision; 0 unverified items reported
Scope compliance	Queries sent outside the authorized zone; out-ofscope items reported	0 and 0
Noise (supporting)	Lines written to the target DNS log	Falls with the query count
Time (supporting only)	Mean of 5 runs	Reported, not used for the conclusion

C. Propose and Implement a Redesign (2 Marks)

Design logic. The cost is spent on guessing, so use known information first, guess only where it is likely to pay off, and never repeat a query. The redesign is a hybrid: passive candidates are filtered, de-duplicated and verified; then a short, bounded guessing step fills the gap.

```
for each unit u:
1   candidates = passive names for u                    # from documents, archive
2   scope filter: drop any name that is not <word>.u.<domain>   # no query to third parties
3   de-duplicate candidates                             # repeats removed
4   verify each remaining candidate with ONE query      # stale ones are removed here
5   guess only the top-30 common words, SKIPPING names already verified   # cache
6   adaptive rule: if the scope has 5 units or fewer, use the full 150-word list in step 5   7   report
only verified items, each with its source
```

Trade-offs introduced. (1) The obscure host (last 10 words of the list) is no longer found when the bounded list is used, so coverage depends on the passive data and on the top-30 list. (2) The method needs passive data; with none, it degrades to bounded guessing. (3) A few extra verification queries are spent on stale candidates (1 per unit). (4) The adaptive rule adds a decision that must be recorded in the authorization.

D. Validate and Compare (2 Marks)

Both versions were run against the same lab zone, the same ground truth and the same scope sizes (5, 25, 125 units). Each version was run 5 times; the number of queries was identical in all 5 runs, so the counts are exact. The query counts were also confirmed from the target DNS log.

Condition	Queries	Lines in target log	Coverage (%)	Precision (%)	Out-of-scope queries / items / unverified items	Time in s: mean (min–max)
5 units · original	750	750	66.7	100.0	0 / 0 / 0	0.208 (0.186–0.252)
5 units · redesign	165	165	83.3	100.0	0 / 0 / 0	0.046 (0.037–0.052)
5 units · redesign (adaptive)	765	765	100.0	100.0	0 / 0 / 0	0.209 (0.181–0.255)
25 units · original	3750	3750	66.7	100.0	0 / 0 / 0	0.889 (0.779–1.097)
25 units · redesign	825	825	83.3	100.0	0 / 0 / 0	0.229 (0.19–0.321)
125 units · original	18750	18750	66.7	100.0	0 / 0 / 0	4.635 (4.535–4.823)

125 units · redesign	4125	4125	83.3	100.0	0 / 0 / 0	1.067 (0.997–1.164)

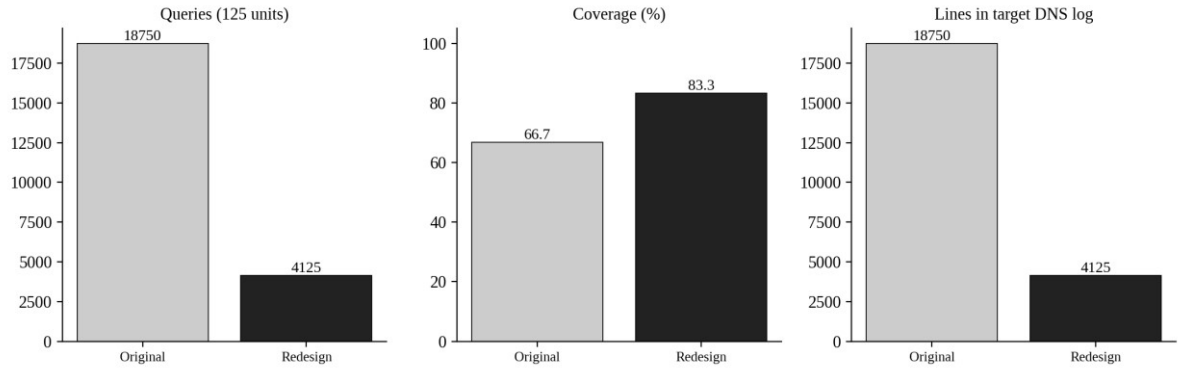


Figure 7: Original and redesigned workflow at 125 units under identical conditions

Does the redesign improve the selected criterion? Yes. At 125 units the queries fell from 18,750 to 4,125 (−78.0 %), the lines written to the target log fell by the same amount, and coverage rose from 66.7 % to 83.3 % (the redesign finds the 2 semi-guessable hosts per unit that the original cannot guess). Precision stayed at 100 %, no out-of-scope query was sent, and the 125 third-party names and 125 stale names in the passive data were removed before reporting (stale names by the verification query). At 25 units the queries fell by 78.0 % and at 5 units by 78.0 %. The target of 70 % fewer queries was met at 25 and 125 units.

Honest limit of the evidence. The lab was designed with a known split of host types and a passive dataset that covers the common and semi-guessable hosts. In a real target the share of hosts that appear in public sources will differ. The result therefore shows that the redesign works when passive data exists; it is not a general guarantee. The time column is supporting evidence only.

E. Defend the Final Engineering Judgment (1 Mark)

Did the redesign meet its objective? Yes for the main objective (large reduction in queries and noise with no loss of coverage and with full scope compliance): 78.0 % fewer queries at 125 units, +16.6 percentage points of coverage, precision 100 %, zero scope violations.

Cases in which the original workflow is still preferable:

- **No passive data.** For a sparse organisation with no public documents (5 units, none available), the original found 66.7 % of hosts, the redesign with the bounded list found only 50.0 %. The original's full word list is what finds the obscure host. The adaptive version (full list for small scopes) restores 66.7 % at the same 750 queries as the original.
- **Small scope where completeness matters more than noise.** At 5 units, original and adaptive redesign cost almost the same (750 against 765 queries), but the adaptive redesign reaches 100 % coverage because it also finds the semi-guessable hosts, so here the redesign is better only by using the full list. If the client wants the least possible change to a trusted procedure, the original remains acceptable.

- **Finding obscure names is the goal.** The bounded redesign misses the obscure host of each unit. If the assessment must find unpublished but guessable names, the full list must be run, and the saving disappears.

Final decision, from the evidence: use the redesigned hybrid as the default for scopes of 25 units or more and for all repeated assessments; use the adaptive full-list variant for scopes of 5 units or fewer; keep the original only when no passive data exists and the scope is small. The decision rests on counted queries, measured coverage and precision, and the target's own log, not on timing or opinion.

Evidence delivered for Q7: diagnosis (A), measurable criteria (B), redesign with ordered steps and trade-offs (C), quantitative comparison under equivalent conditions (D) and the defended judgment (E).

–

Appendix A Documented Command Sequences

Lab start-up and checks named -c named-auth.conf -u root # authoritative server, 127.0.0.1:5300 named -c named-res.conf -u root # caching resolver, 127.0.0.1:5301 python3 whois_server.py # WHOIS fixture server, 127.0.0.1:4343 named-checkzone <zone> zones/<zone>.db # all 13 zone files validated: OK rndc -s 127.0.0.1 -p 9530 -k rndc.key querylog on # count every query received

Collection commands used (Q1, Q2) whois -h 127.0.0.1 -p 4343 leakyco.example

dig @127.0.0.1 -p 5301 leakyco.example TXT +noall +answer dig @127.0.0.1 -p 5300 vpn.leakyco.example A +short dig @127.0.0.1 -p 5300 misconfig.example AXFR nslookup -port=5300 -type=MX shieldco.example 127.0.0.1 dig -b 127.0.0.2 @127.0.0.1 -p 5300 hr-db.split.example A +short # internal viewpoint dig @127.0.0.1 -p 5300 www.sub.deleg.example A +norecurse # referral dig @127.0.0.1 -p 5301 www.cache.example A +short # cached answer dig @127.0.0.1 -p 5300 tsig.example AXFR -y hmac-sha256:xfer-key:<secret> rndc ... reload cache.example IN external ; rndc ... reconfig # after changes

Excerpt of raw output: caching experiment (Q2, case C6) resolver, before change : 192.0.2.80 authoritative after change : 192.0.2.81 resolver immediately after : 192.0.2.80 resolver after 21 s (TTL 20): 192.0.2.81

Excerpt of the target's own DNS query log during the active run (Q6)

```
—
03-Nov-2026 08:08:24.919 client @0x7f8215bb0a78 127.0.0.1#38200
(p4link.u005.bigcorp.example): view external: q
03-Nov-2026 08:08:24.919 client @0x7f8215bb0a78 127.0.0.1#48638
(j6core.u005.bigcorp.example): view external: q
03-Nov-2026 08:08:24.919 client @0x7f8215bb0a78 127.0.0.1#55410
(d1cache.u005.bigcorp.example): view external:
```

Appendix B Scripts, Raw Data and Evidence Bundle

The supporting archive **CO2_Lab_Evidence.zip** contains everything needed to repeat the work and check the numbers. All paths are inside the archive.

File or folder	Purpose
make_lab.py, zones/, named-auth.conf, named-res.conf, whois_server.py	Builds and serves the lab-owned targets (13 zones, views, TSIG key, WHOIS fixtures, document corpus)
labsearch.py	Search engine with site:, filetype:, intitle:, intext: operators over the lab corpus
q1_collect.py, q1_scale.py, q1_analyse.py	Q1 collection, 50-subdomain test and reliability analysis
q2_dnsflow.py	Q2 DNS workflow, seven cases, countermeasure test
ci_workflow.py	Q3 modular competitive-intelligence and awareness workflow
q4_validate.py, q4_order.py	Q4 validator, 24-order evaluation, scalability
q5_traverse.py	Q5 hierarchical traversal and ad-hoc comparison
q6_compare.py, q7_redesign.py	Q6 approach comparison and Q7 redesign validation (real DNS queries)
figs.py	Draws Figures 1 to 7

–

out/predictions_*.json	Predictions written before each run (with file times)
out/raw/q1, out/raw/q2	Raw tool outputs of every dig, nslookup and whois command
out/*_results.json, q1_observed.json, q4_*.json	Collected data and computed results behind every table
out/query.log (compressed)	The lab DNS server's own log of all queries received

Appendix C Submission Requirements Checklist

Requirement	**Where it is satisfied**
Assignment report containing evidence for Q1–Q7	This document; each question ends with an "Evidence delivered" list
Scripts or documented command sequences for all implemented workflows	Appendix A and the scripts in CO2_Lab_Evidence.zip; pseudocode in Q5.2, Q7-C
Student-designed authorized test targets and adversarial cases	Q1.3 (4 targets), Q2.3 (7 cases), Q3.4 (3 organisations), Q4.2 (6 adversarial sequences), Q5.1 (3 structures), Q6.3 and Q7 (scale zone)
Raw collected data and tool outputs	Excerpts in Q1.4, Q2.5, Appendix A; full files in the evidence archive
Prediction-versus-observation comparisons	Q1.5, Q2.4, Q3.6, Q4.3, Q5.3, Q6.5 (mismatches explained)
Tables and graphs wherever they strengthen the evaluation	Figures 1 to 7 and the tables throughout
Source, query, item and scope counts where applicable	Q1.4, Q1.8, Q3.9, Q4.6, Q6.4, Q7-D

–

Design representation or pseudocode for redesigned workflows	Q4.5, Q5.2, Q7-C
Execution time only as supporting evidence	Time is reported but conclusions rest on coverage, reliability, scope compliance and counts